SECURITY ASSESSMENT

ManagerApp

Mobile application security review & remediation plan
Compose Multiplatform (Android + iOS)

Date	5 October 2026
Repository	ManagerApp (branch `login`)
Scope	Client app source: `composeApp` (commonMain / androidMain / iosMain), `iosApp`, Gradle build files, `docs/`
Method	Static source review (manual). No dynamic testing, no server-side testing.
Result	2 High · 7 Medium · 8 Low / hardening findings

1. Executive summary

The app's code structure is clean and avoids several common mistakes: no API keys or secrets are committed, tokens are never written to disk, and the Android attack surface is small. The most serious risk is **transport security**. Every request, **including the login password and the bearer access token**, goes over plain HTTP to a bare IP address, and both platforms have explicit exceptions that allow this. Anyone on the same network (public Wi-Fi, a compromised router, the mobile carrier path) can read or change this traffic.

The second High finding concerns the **AI Sales Report** calls. They send no Authorization header and pass the user's identity as a client-controlled `UserID` parameter. Unless the server checks this independently, one user can read another user's report history (an insecure direct object reference, or IDOR).

2	7	8
High	Medium	Low / hardening

#	Finding	Severity	Effort
H1	All API traffic (credentials, tokens, business data) sent over cleartext HTTP	High	Server + small app change
H2	AI Sales Report endpoints unauthenticated; user identity taken from client-supplied `UserID`	High	Server + small app change
M1	HTTP logging enabled in release builds	Medium	Small
M2	Release build not minified or obfuscated (R8 disabled)	Medium	Small–Medium
M3	`android:allowBackup="true"`	Medium	Small
M4	Incomplete session handling: 401 not validated, tokens never cleared, refresh token unused	Medium	Medium
M5	Geofence attendance enforced only on the client, fails open, no mock-location detection	Medium	Medium
M6	No certificate pinning (applies once HTTPS is enabled)	Medium	Small
M7	Planned token persistence must use Keystore / Keychain	Medium	Medium
L1	Password kept in `LoginUiState` after successful login	Low	Trivial
L2	No screenshot / app-switcher protection on sensitive screens	Low	Small
L3	Exported AI-report PDF/CSV files are unencrypted	Low	Policy
L4	Leftover debug `println`	Low	Trivial
L5	API docs page exposes internal IP, stores userId in localStorage, loads unpinned CDN scripts	Low	Small
L6	Lenient JSON parsing (`isLenient = true`)	Low	Trivial

L7	No automated dependency vulnerability scanning; several libraries behind latest	Low	Small
L8	Device identifier and location collection need privacy disclosure	Low	Policy

2. High severity findings

HIGH H1 · All API traffic is sent over cleartext HTTP

Evidence

- `composeApp/src/commonMain/.../core/network/NetworkConstants.kt:4`: `BASE_URL = "http://210.4.65.222/SupervisorApp/api/v1/"`
- `composeApp/src/androidMain/res/xml/network_security_config.xml:3`: `cleartextTrafficPermitted="true"` for `210.4.65.222`
- `iosApp/iosApp/Info.plist:15`: ATS exception `NSExceptionAllowsInsecureHTTPLoads` for the same IP

Risk. `POST auth/login` sends the login ID and password in plain text, and every later request carries the bearer access token in plain text. An attacker on the network path can capture credentials and tokens, take over sessions, read sales, attendance and distributor data, and change responses (for example, inject false geofence configuration).

Recommendation

- Serve the API over **HTTPS (TLS 1.2+)** on a DNS hostname with a valid CA-issued certificate (for example, Let's Encrypt). Certificates for bare IP addresses are impractical.
- Remove the Android cleartext exception and the iOS ATS exception.
- Configure the base URL per build type so test and production endpoints never get mixed up.

```
<!-- network_security_config.xml -->
<network-security-config>
    <base-config cleartextTrafficPermitted="false" />
</network-security-config>

// NetworkConstants.kt
const val BASE_URL = "https://api.yourdomain.com/SupervisorApp/api/v1/"
```

On iOS, delete the whole `NSAppTransportSecurity` block from `Info.plist`.

HIGH H2 · AI Sales Report endpoints are unauthenticated and trust a client-supplied UserID

Evidence

- `data/remote/aiSalesReport/AISalesReportRemoteDataSource.kt:17-35`: `AskOpenAI`, `AskOpenAIForSummary`, `GetCurrentUserAllQueryLog` and `GetAIFAQs` are called **without** `bearerAuth(...)`. All other data sources attach the token.
- The user's identity is sent as `UserID` (query parameter at lines 28 and 35; request body in `AIReportRequestDto` and `AISummaryRequestDto`).
- A similar pattern exists in `AttendanceRemoteDataSource.kt:33, 41, 73` (`userId` parameter), although those calls do send the token.

Risk. If the server uses the `UserID` it receives, anyone (even without logging in, for the AI endpoints) can change the ID and read another user's prompt history, FAQs and AI-generated sales summaries, or run queries under someone else's identity and data scope. This is an insecure direct object reference (IDOR), and it also lets anyone consume the paid LLM API.

Recommendation

- Server: require a valid bearer token on every endpoint, and **derive the user from the token**. Ignore client-supplied user IDs, or reject them when they don't match the token. This is how `/attendance/team-summary` already works.
- Server: add per-user rate limits on the LLM endpoints.
- App: attach the token in the AI data source, or better, attach it once for all calls (see M4).

```
suspend fun askAI(request: AIReportRequestDto): AIReportResponseDto =
    httpClient.post("AIReportGenerator/AskOpenAI") {
        bearerAuth(tokenStorage.getTokens()?.accessToken ?: "")
        setBody(request)
    }.body()
```

Confirm the server-side behaviour. If the server already checks `UserID` against an authenticated session by some other means, this drops to Medium. Even then, leaving the endpoints unauthenticated is still a problem.

3. Medium severity findings

MEDIUM M1 · HTTP logging is enabled in release builds

Evidence. `core/network/HttpClientFactory.kt:21-23` installs Ktor `Logging` at `LogLevel.INFO` unconditionally.

Risk. Request URLs, including user-ID query parameters, are written to the system log in production. Raising the level to HEADERS or BODY while debugging would also log bearer tokens and passwords, and that change could easily ship.

Recommendation. Install logging only in debug builds, and always sanitize the Authorization header.

```
if (isDebugBuild) {
    install(Logging) {
        level = LogLevel.HEADERS
        sanitizeHeader { it == HttpHeaders.Authorization }
    }
}
```

Provide `isDebugBuild` through a small `expect/actual` (`BuildConfig.DEBUG` on Android, `Platform.isDebugBinary` on iOS) or a Koin property.

MEDIUM M2 · Release build is not minified or obfuscated

Evidence. `composeApp/build.gradle.kts:88`: `isMinifyEnabled = false`. Resource shrinking is also off, and there is no release signing configuration in the build file.

Risk. The APK keeps full class and method names, so reverse-engineering the API contract, business rules (such as the geofence logic) and endpoint paths is trivial. That makes tampering and repackaging easier.

Recommendation

```
getByName("release") {
    isMinifyEnabled = true
    isShrinkResources = true
    proguardFiles(getDefaultProguardFile("proguard-android-optimize.txt"), "proguard-rules.pro")
}
```

Add keep rules for kotlinx-serialization (`@Serializable` classes and their `$serializer`), Ktor and Koin, then run a regression test on a release build. Keep the signing keys outside the repository (CI secrets or `local.properties`).

MEDIUM M3 · Application data can be extracted through backup

Evidence. `composeApp/src/androidMain/AndroidManifest.xml:10`: `android:allowBackup="true"`.

Risk. SharedPreferences (login ID today, and tokens once M7 is done) can be copied by `adb backup` on older devices, or included in cloud backups and restored onto another device.

Recommendation

```
<application
    android:allowBackup="false"
    android:dataExtractionRules="@xml/data_extraction_rules"
    android:fullBackupContent="false" ... >
```

If backup is needed, exclude credential stores explicitly with `<exclude domain="sharedpref" path="..."/>`.

MEDIUM M4 · Session handling gaps (401 detection, token clearing, refresh)

Evidence

- `HttpClientFactory.kt` sets neither `expectSuccess = true` nor an `HttpResponseValidator`, so `SafeApiCall.kt:68` (`is ClientRequestException`) never triggers. A 401 response surfaces as a deserialization or business error, not as `AppError.Unauthorized`.
- `TokenStorage.clearTokens()` and `UserSession.clearUser()` are never called. "Session expired" navigates to Login but leaves the old token in memory.
- The refresh token is parsed and stored but never used.
- Each data source attaches the token by hand, which is how H2 happened.

Risk. Expired or revoked sessions may not be detected reliably. The previous user's token stays usable inside the process after logout or expiry. Manual per-call auth makes it easy to miss a call.

Recommendation. Centralise authentication with Ktor's `Auth` plugin.

```
install(Auth) {
    bearer {
        loadTokens { tokenStorage.getTokens()?.let { BearerTokens(it.accessToken, it.refreshToken)
} }
        refreshTokens { /* call refresh endpoint; on failure clear storage + emit logout */ }
        sendWithoutRequest { request -> !request.url.encodedPath.endsWith("auth/login") }
    }
}
expectSuccess = true  // so 4xx/5xx reach SafeApiCall's ClientRequestException branch
```

Call `tokenStorage.clearTokens()` and `userSession.clearUser()` on logout and on any unrecoverable 401. Then remove the per-call `authorize(token)` helpers.

MEDIUM M5 · Geofence attendance can be bypassed

Evidence. `domain/usecase/attendance/CheckGeofenceUseCase.kt:33-39` decides on the device whether the user is inside the distributor radius. It returns `Allowed` whenever the distributor location, config or radius is missing. No mock-location check is done in `AndroidLocationProvider` or `IosLocationProvider`.

Risk. A fake-GPS app, a rooted or jailbroken device, or a modified APK can mark attendance from anywhere. Because the check fails open, any gap in configuration silently disables it.

Recommendation

- Repeat the distance check on the **server** in `attendance/save-attendance` using the submitted coordinates. The client check is for UX only.
- Reject or flag mock locations on Android: `location.isMock` (API 31+), or `location.isFromMockProvider` (older). Send a `isMockLocation` flag with the attendance request.
- For stronger assurance, add Play Integrity (Android) or App Attest (iOS) tokens to attendance submissions.
- Decide explicitly whether missing configuration should block check-in or be logged for review, rather than allowing it silently.

MEDIUM M6 · No certificate pinning

Evidence. The OkHttp engine (`PlatformModule.android.kt`) and the Darwin engine (`PlatformModule.ios.kt`) are created without pinning.

Risk. Once HTTPS is in place (H1), a user-installed or corporate CA could still intercept traffic. This matters most for a field-sales app used on unmanaged devices.

Recommendation. Pin the server's public key, and include a backup pin so certificates can rotate.

```
OkHttp.create {
    config {
        certificatePinner(
            CertificatePinner.Builder()
                .add("api.yourdomain.com", "sha256/PRIMARY_PIN=", "sha256/BACKUP_PIN=")
                .build()
        )
    }
}
```

On iOS, use `Darwin.create { handleChallenge { ... } }` to compare the server's public-key hash.

MEDIUM M7 · Planned token persistence must use secure storage

Evidence. `InMemoryTokenStorage.kt` and `InMemoryUserSession.kt` are documented placeholders that will later be replaced with persistent storage ("SQLDelight/secure-storage"). The login ID is already stored in plain `SharedPreferences` (`AndroidLoginPreferences.kt:9`) and `NSUserDefaults`.

Risk. Storing access or refresh tokens in SharedPreferences, NSUserDefaults or an unencrypted SQLDelight database would expose them to backup extraction (M3), rooted devices and forensic tools.

Recommendation. Implement `TokenStorage` with Android Keystore-backed encryption (for example, encrypting a DataStore value with a Keystore AES-GCM key), and with the **iOS Keychain** (`kSecAttrAccessibleAfterFirstUnlockThisDeviceOnly`). Provide both through the platform Koin modules, as the project conventions require. Never put tokens in the SQLDelight database.

4. Low severity / hardening

LOW L1 · Password kept in UI state after login

`LoginViewModel.kt:57-65`: `LoginUiState.password` is never cleared after a successful login. The ViewModel is activity-scoped, so the plaintext password stays in memory for the whole session. **Fix:** `_uiState.update { it.copy(password = "") }` before `NavigateToHome`.

LOW L2 · No screenshot / recent-apps protection

Sales figures, team data and the login screen can be captured by screenshots, screen recorders and the recent-apps thumbnail. **Fix:** on Android, set `window.setFlags(FLAG_SECURE, FLAG_SECURE)` in `MainActivity` (or only on sensitive routes). On iOS, add a blur or overlay view in `sceneWillResignActive`.

LOW L3 · Exported reports are unencrypted

AI Sales Report PDF and CSV exports (FileKit) are saved wherever the user chooses, without protection. That's acceptable for a user-initiated export, but confirm it with the business and consider a watermark ("Confidential · user · date") on PDFs.

LOW L4 · Leftover debug output

`navigation/AppNavigation.kt:65`: `println("Login page")`. Remove it, and route any future logging through a debug-only logger.

LOW L5 · API documentation page

`docs/index.html`:

- Calls the test server over HTTP.
- Saves the logged-in `userId` to `localStorage` (line 38).
- Loads Swagger UI from `unpkg.com` with a floating `@5` version and no Subresource Integrity (lines 6, 23, 24).

Together with `docs/openapi.yaml`, it reveals the internal IP and the full API surface.

Fix:

- Keep the docs off public hosting.
- Pin an exact Swagger UI version and add `integrity` / `crossorigin` attributes.
- Use `sessionStorage`, and point the docs at the HTTPS host once H1 is fixed.

LOW L6 · Lenient JSON parsing

`core/di/NetworkModule.kt:12`: `isLenient = true` accepts malformed JSON such as unquoted keys. Keep `ignoreUnknownKeys`, but turn off `isLenient` unless the server really sends non-standard JSON.

LOW L7 · Dependency hygiene

`gradle/libs.versions.toml` pins Ktor 3.0.3, Koin 4.0.2 and kotlinx-serialization 1.7.3, which are behind current releases. No automated vulnerability scanning was found. (No specific CVE was checked as part of this review.)

Fix: enable Renovate or Dependabot on `libs.versions.toml`, add the OWASP Dependency-Check Gradle plugin (or GitHub dependency review) to CI, and update Ktor and OkHttp regularly, since they handle all network input.

LOW L8 · Privacy disclosure

Login and attendance send `ANDROID_ID` (`AndroidDeviceInfoProvider.kt:10`), the OS version and the precise location. Make sure the privacy policy, the Google Play Data Safety form and the iOS privacy manifest declare these, and collect location only when marking attendance (the current behaviour).

5. What is already done well

- No hardcoded API keys, secrets or credentials in the source or build files. `local.properties` is git-ignored.
- Access and refresh tokens are kept only in memory and never written to disk.
- Small Android attack surface: only the launcher `MainActivity` is exported, and only the INTERNET and LOCATION permissions are requested.
- Cleartext exceptions are limited to one host (`includeSubdomains="false"`) rather than enabled globally.
- The password field is masked by default. Errors are mapped to user-friendly messages, so stack traces don't leak to the UI.
- A single, consistent error-handling path (`safeApiCall` → `AppError`) makes the central fixes for M4 easy to apply.

6. Remediation roadmap

When	Actions
Now (before production)	H1: HTTPS on a hostname, remove the cleartext / ATS exceptions. H2: authenticate the AI endpoints and derive the user from the token on the server. M1: logging in debug builds only. M3: `allowBackup="false"`. L1, L4: quick code fixes.
Next sprint	M4: Ktor `Auth` bearer plugin + `expectSuccess` + token clearing on logout / 401. M2: R8 with keep rules; test the release build. M5: server-side geofence validation and mock-location rejection. M6: certificate pinning, once HTTPS is live.
Later	M7: Keystore / Keychain `TokenStorage` when persistent login is added. L2: FLAG_SECURE / snapshot blur. L5–L8: docs hardening, dependency scanning in CI, privacy declarations, export watermarking. Consider Play Integrity / App Attest for attendance.

7. Method and limitations

- Manual static review of the whole client repository: Kotlin sources in every source set, the Android manifest and resources, iOS `Info.plist` and xcconfig, Gradle build and version catalog, and `docs/`.
- Not covered: the backend (`210.4.65.222`), dynamic or runtime testing, binary analysis of a release APK or IPA, and third-party library CVE lookup.
- The iOS target could not be built on the review machine (Windows), so iOS findings come from source review only.
- Several findings (H2, M5) depend on server behaviour and should be confirmed with the backend team.